

Projekt Dyplomowy 2026 – raport postępu prac nr 2

Wyznaczanie poziomów Fibonacciego pod kątem możliwego zastosowania
w modelach klasyfikacyjnych.

Paweł Myszka
nr albumu: 331720

26 maja 2026

Spis treści

1	Cel projektu	2
2	Przegląd literatury i podstawy teoretyczne	2
3	Postęp prac – status etapów	3
4	Opis nowych etapów realizacji	3
4.1	Selekcja cech – redukcja wymiarowości	3
4.2	Optymalizacja XGBoost – scale_pos_weight	4
4.3	Sieć neuronowa MLP (PyTorch)	5
4.4	Analiza wpływu parametrów etykietowania na jakość modeli	5
5	Wyniki modeli klasyfikacyjnych	6
5.1	Wyniki dla konfiguracji bazowej (baseline)	6
5.2	Wyniki dla Wariantu 1 (konfiguracja finalna)	6
5.3	Porównanie wariantów etykietowania	7
6	Wnioski z dotychczasowych prac	7
7	Planowane dalsze kroki	7
7.1	Backtesting strategii transakcyjnej	7
8	Podsumowanie	8

1 Cel projektu

Celem projektu jest zbadanie, czy poziomy Fibonacciego wyznaczone na podstawie lokalnych ekstremów cenowych mogą stanowić użyteczne cechy wejściowe dla modeli klasyfikacyjnych w kontekście analizy technicznej rynków finansowych. Projekt ma na celu stworzenie systemu automatycznego wyznaczania stref wsparcia i oporu opartych na geometrii Fibonacciego, a następnie klasyfikację tych stref jako *trafione* lub *nietrafione* przez algorytmy uczenia maszynowego.

2 Przegląd literatury i podstawy teoretyczne

Podstawę teoretyczną projektu stanowią narzędzia analizy technicznej wywodzące się z ciągu Fibonacciego oraz złotego podziału. Kluczową pozycją literaturową jest:

Paweł Danielewicz, *Geometria Fibonacciego* – literatura poświęcona zastosowaniu współczynników Fibonacciego w analizie rynków finansowych. Autor szczegółowo omawia konstruowanie stref wsparcia i oporu oraz interpretację zachowania ceny w okolicach kluczowych poziomów (0,236; 0,382; 0,5; 0,618; 0,786; 1,0; 1,272; 1,618 i inne).

W projekcie zastosowano rozszerzony zestaw poziomów Fibonacciego zgodny z podejściem prezentowanym przez Danielewicza:

$L = 0; 0,236; 0,278; 0,38; 0,44; 0,5; 0,577; 0,61; 0,667; 0,707; 0,77; 0,786; 0,8; 0,878; 0,941; 1,0; 1,13; 1,272; 1,414; 1,618; 1,732; 2,058; 2,236; 2,618$

3 Postęp prac – status etapów

Tabela 1: Status realizacji poszczególnych etapów projektu

Etap	Status	Uwagi
Pobranie i oczyszczenie danych cenowych	Ukończony	30 tickerów, 10 lat, 2514 sesji/ticker
Wyznaczenie lokalnych minimów i maksimów	Ukończony	Algorytm <code>argrelextrema</code>
Wyznaczanie trendów wzrostu/spadku	Ukończony	Sekwencje swing high/low
Wyznaczenie poziomów Fibonacciego	Ukończony	24 poziomy
Generowanie stref wsparcia i oporu	Ukończony	Strefy agregowane
Wizualizacja wyników	Ukończony	Wykresy świecowe z oznaczeniami
Tworzenie zbioru danych – etykietowanie stref	Ukończony	Etykiety: trafiona / nietrafiona
Wyznaczenie cech wejściowych	Ukończony	22 cechy (RSI, ATR, MACD, BB, ...)
Selekcja cech (redukcja wymiarowości)	Ukończony	14 cech po usunięciu najmniej ważnych
Normalizacja / standaryzacja danych	Ukończony	<code>StandardScaler</code> dla SVM/LR
Podział danych treningowy/testowy	Ukończony	Podział chronologiczny 80/20
Oversampling (SMOTE)	Ukończony	<code>sampling_strategy=0.5</code>
Trening i porównanie modeli klasyfikacyjnych	Ukończony	RF, DT, SVM, LR, XGBoost, MLP
Optymalizacja progu decyzyjnego	Ukończony	<code>scale_pos_weight</code> , próg z walidacji
Analiza ważności cech	Ukończony	Wykres <i>feature importance</i>
Analiza wpływu parametrów etykietowania	Ukończony	Porównanie 3 wariantów etykietowania
Sieć neuronowa MLP (PyTorch)	Ukończony	<code>BCEWithLogitsLoss</code> , 80 epok
Backtesting strategii transakcyjnej	Planowany	

4 Opis nowych etapów realizacji

Raport 2 opisuje prace wykonane po złożeniu raportu 1. Etapy zrealizowane wcześniej (pobranie danych, wyznaczanie swing pointów, trendów, poziomów Fibonacciego, generowanie i etykietowanie stref, wyznaczanie cech) zostały szczegółowo opisane w raporcie 1 i nie są tu powtarzane. W tym raporcie opisano wyłącznie nowe działania.

4.1 Selekcja cech – redukcja wymiarowości

Na podstawie analizy ważności cech wyznaczonej przez model Random Forest (tabela 2) usunąłem 10 cech o najniższej ważności (poniżej progu 0,02):

- ilosc_w_klastrze (0,010),
- dlugosc_trendu (0,012),
- typ_trendu (0,012),
- trend_sma (0,013),
- wolumen_w_trendzie (0,017),
- atr_wejscie (0,017),
- wolumen_sredni (0,017),
- ile_razy_testowany (0,017),
- rsi_strefa (0,017),
- macd_hist (0,019),
- vol_relatywny (0,019).

Po selekcji zbiór cech wejściowych liczy **12 cech**. Trzy cechy o najwyższej ważności to: odl_od_swing_low (0,222), odleglosc_od_strefy (0,147) oraz bb_pozycja (0,112).

Tabela 2: Ważność cech wejściowych – Random Forest

Rank	Cecha	Ważność
1	odl_od_swing_low	0,222
2	odleglosc_od_strefy	0,147
3	bb_pozycja	0,112
4	odl_od_swing_high	0,084
5	poziom_fib	0,054
6	odleglosc_od_sma20	0,066
7	zmiennosc_trendu	0,040
8	zmiana_trendu	0,030
9	rsi_wejscie	0,027
10	bb_szerokosc	0,025
11	momentum5	0,022
<i>Cechy usunięte (ważność < 0,02):</i>		
–	vol_relatywny	0,019
–	macd_hist	0,019
–	ile_razy_testowany	0,017
–	rsi_strefa	0,017
–	wolumen_sredni	0,017
–	atr_wejscie	0,017
–	wolumen_w_trendzie	0,017
–	trend_sma	0,013
–	typ_trendu	0,012
–	dlugosc_trendu	0,012
–	ilosc_w_klastrze	0,010

4.2 Optymalizacja XGBoost – scale_pos_weight

W celu bezpośredniego adresowania nierównowagi klas zastosowano parametr `scale_pos_weight` modelu XGBoost, definiowany jako stosunek liczby próbek klasy większościowej do klasy mniejszościowej:

$$\text{scale_pos_weight} = \frac{|\text{nietrafiona}|}{|\text{trafiona}|} \approx 10,31$$

Parametr ten informuje model, że błąd na klasie mniejszościowej jest ponad 10-krotnie kosztowniejszy niż błąd na klasie większościowej. Zastosowano następującą konfigurację modelu:

- `n_estimators=400`, `max_depth=6`, `learning_rate=0.03`,
- `subsample=0.8`, `colsample_bytree=0.8`,
- `min_child_weight=5`, `gamma=1`,
- `scale_pos_weight=10.31`.

Próg decyzyjny wyznaczono na zbiorze walidacyjnym (ostatnie 20% danych treningowych) poprzez optymalizację krzywej Precision-Recall, unikając tym samym wycieku danych (*data leakage*) przy wyborze progu.

4.3 Sieć neuronowa MLP (PyTorch)

Zaimplementowano wielowarstwową sieć neuronową (ang. *Multi-Layer Perceptron*) w bibliotece PyTorch. Architektura sieci:

- warstwa wejściowa: 14 cech,
- warstwa ukryta 1: 256 neuronów, BatchNorm, ReLU, Dropout(0.1),
- warstwa ukryta 2: 128 neuronów, BatchNorm, ReLU, Dropout(0.1),
- warstwa ukryta 3: 64 neurony, BatchNorm, ReLU, Dropout(0.1),
- warstwa wyjściowa: 1 neuron (logit).

Jako funkcję straty zastosowano BCEWithLogitsLoss z parametrem `pos_weight=10.31`, co jest odpowiednikiem `class_weight='balanced'` dla sieci neuronowych. Model trenowano przez 200 epok. (`lr=0.001`, `weight_decay=1e-4`).

4.4 Analiza wpływu parametrów etykietowania na jakość modeli

Przeprowadzono systematyczne badanie wpływu parametrów etykietowania stref na stosunek klas i wynikową jakość modeli. Porównano trzy warianty:

Tabela 3: Porównanie wariantów etykietowania stref

Wariant	tolerancja	min_odbicie	n_świec	trafione	stosunek
Bazowo	0,02	0,03	5	3 455	10,6:1
Wariant 1	0,02	0,02	10	5 459	6,3:1
Wariant 3	0,02	0,03	10	4 779	7,6:1

Wariant 1 (`min_odbicie=0.02`, `n_świec=10`) przyjęto jako finalną konfigurację etykietowania. Zmiana ta oznacza, że strefa jest uznawana za “trafioną”, jeśli po dotknięciu jej przez cenę nastąpiło odbicie co najmniej 2% w ciągu 10 kolejnych świec (zamiast 3% w ciągu 5 świec w konfiguracji bazowej). Parametr ten pozostaje ekonomicznie sensowny – 2% ruch cenowy w horyzoncie 2 tygodni handlowych stanowi realny sygnał rynkowy.

5 Wyniki modeli klasyfikacyjnych

5.1 Wyniki dla konfiguracji bazowej (baseline)

Tabela 4 przedstawia wyniki wszystkich modeli dla konfiguracji bazowej (stosunek klas 10,6:1).

Tabela 4: Porównanie modeli – konfiguracja bazowa (min_odbicie=0,03, n_świec=5)

Model	F1	Recall	Precision	Accuracy
Drzewo decyzyjne	0,323	0,387	0,277	0,872
Regresja logistyczna	0,388	0,799	0,256	0,801
XGBoost + scale_pos_weight próg 0,5	0,410	0,872	0,268	0,802
SVM	0,423	0,754	0,294	0,838
XGBoost	0,435	0,416	0,456	0,915
MLP próg 0,5	0,445	0,723	0,321	0,858
MLP próg 0,68	0,457	0,580	0,377	0,891
Random Forest	0,480	0,602	0,399	0,897
XGBoost + scale_pos_weight próg 0,63	0,504	0,717	0,388	0,889

5.2 Wyniki dla Wariantu 1 (konfiguracja finalna)

Tabela 5 przedstawia wyniki wszystkich modeli dla konfiguracji finalnej (stosunek klas 6,3:1).

Tabela 5: Porównanie modeli – Wariant 1 (min_odbicie=0,02, n_świec=10)

Model	F1	Recall	Precision	Accuracy
Drzewo decyzyjne	0,509	0,554	0,471	0,860
Regresja logistyczna	0,557	0,855	0,413	0,822
XGBoost + scale_pos_weight próg 0,5	0,586	0,888	0,437	0,836
SVM	0,586	0,855	0,445	0,842
MLP próg 0,5	0,624	0,837	0,497	0,868
MLP próg 0,60	0,635	0,767	0,542	0,885
XGBoost	0,637	0,643	0,632	0,904
Random Forest	0,645	0,749	0,566	0,892
XGBoost + scale_pos_weight próg 0,62	0,648	0,782	0,553	0,889

5.3 Porównanie wariantów etykietowania

Tabela 6: Najlepszy model (XGBoost + scale_pos_weight) w zależności od wariantu etykietowania

Wariant	Stosunek klas	F1	Recall	Precision
Baseline	10,6:1	0,504	0,717	0,388
Wariant 1	6,3:1	0,648	0,782	0,553
Wariant 3	7,6:1	0,602	0,750	0,503

Zmiana parametrów etykietowania z konfiguracji bazowej na Wariant 1 spowodowała wzrost F1-score najlepszego modelu o +0,144 (z 0,504 do 0,648). Jest to wynik istotny: **definicja etykiety “trafiona strefa” okazała się mieć większy wpływ na skuteczność klasyfikacji niż wybór architektury modelu.**

6 Wnioski z dotychczasowych prac

1. **Nierównowaga klas jest kluczowym problemem.** Stosunek klas 10,6:1 w konfiguracji bazowej powoduje, że wszystkie modele osiągają wysoką dokładność ogólną (*accuracy*) przy jednocześnie niskich wartościach F1-score dla klasy mniejszościowej. Obawiam się, że żadna technika modelowania nie zastąpi lepszych danych wejściowych.
2. **Parametry etykietowania mają decydujące znaczenie.** Zmiana minimalnego odbicia z 3% na 2% i wydłużenie okna obserwacji z 5 do 10 świec zmniejszyło nierównowagę klas z 10,6:1 do 6,3:1 i poprawiło F1 o 28% (z 0,504 do 0,648).
3. **XGBoost z wagą klasy mniejszościowej jest najlepszym modelem.** We wszystkich wariantach etykietowania XGBoost z parametrem `scale_pos_weight` i optymalnym progiem decyzyjnym przewyższa pozostałe modele.
4. **MLP nie przewyższa metod gradientowego boostingu.** Architektura MLP (PyTorch, 200 epok, BCEWithLogitsLoss) osiągnęła F1=0,635 w Wariacie 1, co jest wynikiem porównywalnym z Random Forest (0,645), lecz niższym od XGBoost (0,648). Potwierdza to obserwacje z literatury, że na danych tabelarycznych metody gradientowego boostingu są trudne do pobicia przez sieci neuronowe.

7 Planowane dalsze kroki

7.1 Backtesting strategii transakcyjnej

Planowane jest przeprowadzenie backtestingu hipotetycznej strategii transakcyjnej opartej na sygnałach modelu XGBoost. Strategia zakłada:

- otwarcie pozycji przy każdym sygnale “trafiona” wystawionym przez model,
- zamknięcie pozycji po osiągnięciu zadanego celu zysku lub stop-loss,

Backtesting umożliwi ocenę praktycznej użyteczności modelu niezależnie od metryk klasyfikacyjnych (F1, precision, recall).

8 Podsumowanie

W ramach drugiego etapu prac zrealizowano następujące działania: redukcję wymiarowości ($22 \rightarrow 14$ cech), optymalizację XGBoost przez parametr `scale_pos_weight`, implementację sieci neuronowej MLP w PyTorch oraz systematyczną analizę wpływu parametrów etykietowania na jakość modeli.

Najlepszy osiągnięty wynik to **F1 = 0,648** (XGBoost + `scale_pos_weight`, próg 0,62) dla Wariantu 1 etykietowania. Stanowi to poprawę o +0,144 względem najlepszego wyniku z raportu 1 (F1 = 0,504). Kolejnym i ostatnim planowanym etapem jest backtesting strategii transakcyjnej.